CHANDPUR SCIENCE AND TECHNOLOGY UNIVERSITY
Department of Computer Science and Engineering



Cropora

A Hybrid Cloud and On-Device Plant Disease Diagnosis App for Android with English–Bangla Guidance

PROJECT REPORT

CSE 2206 • Mobile Application Development Project

October 2026

SUBMITTED BY

Mahir Faysal Tusher

ID: B220101009

B.Sc. in Computer Science and Engineering
Session 22-23

SUPERVISED BY

Prince Mahmud

Assistant Professor, Department of CSE
Chandpur Science and Technology University

BUILT WITH

Kotlin · TensorFlow Lite · FastAPI · Room · MobileNetV2

github.com/M-F-Tushar/Cropora

CROPORA PROJECT REPORT

Table of Contents

List of Figures iii

List of Tables iv

Abstract v

Acknowledgement vi

1 Introduction 1

1.1 Background Study . . . 1
1.2 Problem Statement . . . 2
1.3 Objectives of the Project . . . 2
1.4 Scope of the Study . . . 3
1.5 Contributions . . . 3
1.6 Organisation of the Report . . . 4

2 Research and Literature Review 5

2.1 Evolution of Plant Disease Diagnosis . . . 5
2.2 Datasets: PlantVillage . . . 5
2.3 Cloud versus On-Device Inference . . . 6
2.4 Android, Kotlin and Jetpack . . . 6
2.5 MobileNetV2, TensorFlow Lite and FastAPI . . . 7
2.6 Related Systems . . . 7

3 Research Methodology 8

3.1 Development Methodology . . . 8
3.2 Requirements . . . 8
3.3 System Architecture . . . 9
3.4 Machine Learning Pipeline . . . 10
3.5 Backend Design (FastAPI) . . . 13
3.6 Android Application Design . . . 14

4 Results and Discussion 18

4.1 Implementation at a Glance . . . 18
4.2 Normal Mode . . . 18
4.3 Advanced Mode . . . 19
4.4 Saved Scans from the Test Phone . . . 21

4.5 Keras versus TFLite Parity . . . 22
4.6 Inference Latency . . . 22
4.7 Backend Verification Evidence . . . 22
4.8 Disease Taxonomy . . . 23

5 Testing and Verification **24**
5.1 Test Strategy and Environment . . . 24
5.2 Unit and Pipeline Verification . . . 24
5.3 Backend Tests and Known Issue . . . 25
5.4 Instrumented and Manual Verification . . . 25

6 Conclusion and Future Work **26**
6.1 Summary . . . 26
6.2 Limitations and Known Issues . . . 26
6.3 Future Work . . . 27
6.4 Final Remarks . . . 27

References **28**

A Repository Structure and Reproduction **30**

B Guidance Sources **31**

C The 38 Model Classes **32**

List of Figures

1.1 Roadmap of the report 4

3.1 Cropora system architecture 9
3.2 Dual-engine scan flow 10
3.3 Original workflow sketch of the Cropora design 10
3.4 Layer structure of the deployed model 11
3.5 Inference contract pipeline 11
3.6 Model file size before and after TFLite conversion 12
3.7 Swagger UI of the Cropora backend 14
3.8 Simplified navigation map 15
3.9 Room database schema 16
3.10 Colour tokens from application resources 17

4.1 Normal mode main flow 19
4.2 Normal mode supporting screens 19
4.3 Annotated Advanced-mode home screen 20
4.4 Advanced mode workflow and screens 20
4.5 Advanced mode insights and settings 21
4.6 Desktop CPU inference time comparison 22
4.7 Backend verification evidence 23
4.8 Disease and healthy classes per crop 23



List of Tables

1.1 Scope of the Cropora project 3

2.1 Comparison of plant-diagnosis app types with Cropora 7

3.1 Main functional and non-functional requirements 8
3.2 Model artefacts 11
3.3 Inference contract rules and enforcement 12
3.4 Backend endpoints and response semantics 13
3.5 Backend configuration environment variables 14
3.6 Main Android dependencies in app build configuration 15

4.1 Implementation summary 18
4.2 Comparison of presentation modes 21
4.3 Selected entries visible in the scan history 21
4.4 Measured Keras–TFLite parity on synthetic fixtures 22
4.5 Classes per crop 23

5.1 Test suites and execution outcomes 24

6.1 Objectives and project outcomes 26

A.1 Main repository directories 30

B.1 Primary agricultural extension sources 31



CROPORA PROJECT REPORT

Abstract

Foliar plant diseases threaten crop yields, but farmers often lack timely expert advice. Deploying an image classifier on a phone also raises practical problems: weak connectivity, inconsistent preprocessing, overconfident predictions, and class labels without useful guidance in the user's language.

Cropora is a native **Kotlin** Android application that classifies leaf photographs into **38 PlantVillage classes across 14 crops** (26 disease conditions and 12 healthy classes). Users choose an **online FastAPI/Keras engine** or a fully offline **TensorFlow Lite engine**. Conversion reduced the model from 25.14 MB to 9.09 MB (63.9%). The classifier is a *third-party*, pre-trained MobileNetV2 model; this project's contribution is its validated integration, conversion and deployment, not training a new network.

Both engines share an inference contract: 224×224 RGB input with float values in $[0, 255]$, in-graph rescaling to $[-1, 1]$, validated 38-way probabilities, and an uncertainty flag controlled by a user-adjustable threshold (default 50%, range 5–95%). A 38-entry English–Bangla catalog provides symptoms, recommended actions and prevention advice, synthesised from extension sources cited per entry. Normal and Advanced presentation modes, custom charts, saved scans with optional location, daily reminders and per-app language switching complete the mobile workflow.

Verification for this report found 12/12 model-pipeline tests passing, including top-1 Keras–TFLite agreement on three synthetic fixtures with maximum confidence difference below 10^{-6}. All 12 Android JVM tests passed and a debug APK was built. The backend had 19 passing tests, one TensorFlow-dependent skip and one known loader-fallback failure with six failing sub-cases; 24 instrumented tests were not re-run. These results verify engineering contracts, not field accuracy or on-phone latency.

38	14	2	9.09	2
disease & healthy classes	crop species	inference engines	MB on-device model	languages (EN / BN)

Keywords: plant disease detection, Android, Kotlin, TensorFlow Lite, FastAPI, MobileNetV2, on-device inference, localisation, Room database.



CROPORA PROJECT REPORT

Acknowledgement

I would like to express my sincere gratitude to **Prince Mahmud**, Assistant Professor, Department of Computer Science and Engineering, **Chandpur Science and Technology University (CSTU)**, for his supervision, technical feedback and encouragement throughout the design, implementation and verification of Cropora.

I thank all faculty members of the Department of Computer Science and Engineering at CSTU for the coursework in mobile application development, databases, artificial intelligence and software engineering on which this project builds.

I acknowledge the open-source community whose work made this project possible: the authors of the PlantVillage dataset, the author of the pre-trained `Plant-Disease-Detector` model (Muhammad Hassan) that Cropora deploys, and the maintainers of TensorFlow, FastAPI, Retrofit and the Android Jetpack libraries. I am also grateful to the cooperative extension services—University of Minnesota Extension, Clemson Cooperative Extension, NC State Extension, UC IPM, UF/IFAS, the Pacific Northwest Plant Disease Management Handbook, University of Kentucky and USDA APHIS—whose public fact sheets are the basis of the guidance catalog.

Finally, I thank my family and friends for their patience and support throughout this semester.

Mahir Faysal Tusher
ID: B220101009
Department of Computer Science and Engineering
Chandpur Science and Technology University

CHAPTER 1

1 Introduction

IN THIS CHAPTER

1.1	Background Study	1	1.4	Scope of the Study	3
1.2	Problem Statement	2	1.5	Contributions	3
1.3	Objectives of the Project	2	1.6	Organisation of the Report	4

1.1 Background Study

Agriculture is the foundation of food security and rural income in Bangladesh and across South Asia, where most farms are small and a single failed harvest can be severe for a household. Plant diseases caused by fungi, bacteria, oomycetes, viruses and pests are a major driver of pre-harvest loss. A global expert survey by Savary *et al.* estimated yield losses from pathogens and pests of roughly 17–30% for five staple crops (wheat, rice, maize, potato and soybean), with the highest losses in food-deficit regions[1]. Early and correct identification of leaf symptoms lets a grower remove infected material, change irrigation and spacing, and seek local advice before a disease spreads through the field.

Traditionally, diagnosis relies on visual inspection by an extension officer or on laboratory methods such as microscopy and PCR. These are accurate but slow, costly and often out of reach for rural growers. Over the last decade, smartphone cameras combined with convolutional neural networks (CNNs)—enabled by public datasets such as PlantVillage[2, 3] and efficient mobile architectures such as MobileNetV2[5]—have made automated, image-based disease recognition practical on a phone.

Accuracy alone, however, does not make a dependable mobile tool. Connectivity in rural fields is often weak or absent, requiring an offline fallback; moving a model between Python and Kotlin risks silent preprocessing skew; softmax classifiers overconfidently guess on un-decodable photos; and raw scientific labels lack actionable guidance or native language accessibility. **Cropora** was built to address these engineering gaps: a native Android application in Kotlin, supported by a FastAPI backend and an on-device TensorFlow Lite engine, identifying 38 foliar health states across 14 crops with

bilingual guidance in English and Bangla.

1.2 Problem Statement

Users who try to diagnose leaf diseases with typical mobile tools face five related problems:

1. **Dependence on connectivity.** Many diagnosis apps are thin clients for a remote API; when the upload fails in a field with poor reception, the user gets no answer.
2. **Preprocessing drift between server and phone.** A MobileNetV2-based Keras model may contain its own rescaling layer ($x/127.5 - 1$) and therefore expects raw $[0, 255]$ inputs. If the mobile code normalises pixels to $[0, 1]$ a second time, the model still runs but its predictions become wrong—without any error message.
3. **Overconfident answers.** A softmax output always sums to one, so an unclear photo still produces a "top" class. Without an explicit uncertainty rule, guesses look like diagnoses.
4. **Labels without guidance.** A model label is not actionable unless it is linked to symptoms, safe management steps and prevention advice from credible sources.
5. **Language and interface fit.** A single English interface does not serve first-time rural users who prefer Bangla and a simple flow, nor power users who want history, filters and charts.

1.3 Objectives of the Project

The goal is to design, implement and verify Cropora, a bilingual Android app for leaf disease diagnosis that works both online and offline. The specific objectives are:

O1. Provide **two inference engines**—a FastAPI service (`POST /predict`) and an on-device TensorFlow Lite classifier—selectable by the user on the scan screen.

O2. Define and enforce a **38-class inference contract** in Python and Kotlin: fixed label order, input shape $[1, 224, 224, 3]$, output shape $[1, 38]$, rescaling inside the model graph, validated probabilities, and an uncertainty flag below a configurable threshold.

O3. Convert the Keras model to TensorFlow Lite and **prove numerical parity** between the two formats with automated tests.

O4. Bundle a **38-entry guidance catalog** (symptoms, what to do, prevention) in English and Bangla, with per-entry source links, available offline.

O5. Offer **Normal and Advanced presentation modes**, with custom chart views in Advanced mode.

O6. Support **English and Bangla** through Android per-app language settings.
O7. Store scans in a **Room database** with a tested schema migration, optional location tagging, sharing and deletion.
O8. Build an **automated test suite** covering the model pipeline, backend API and Android logic and UI.

1.4 Scope of the Study

Table 1.1: Scope of the Cropora project (release 0.1.0)

Dimension	In scope (implemented)	Out of scope / future
Platform	Native Android app (Kotlin 1.9.22, minSdk 24 / Android 7.0 to targetSdk 34 / Android 14).	iOS client, web dashboard, user accounts and cloud sync.
Classes	38 PlantVillage classes across 14 crops (26 disease conditions, 12 healthy).	Other crops, nutrient deficiencies, pest counting, severity grading.
Model	Integration of a pre-trained third-party MobileNetV2 model; Keras on the server, TFLite on the phone; mock predictor for development.	Training or fine-tuning a new model; field-image evaluation; quantisation.
Guidance	Cultural and sanitation guidance with source links, in English and Bangla.	Pesticide doses, product recommendations, agronomist-certified advice.
Data	Local Room database (`cropora.db` v2) of scan results with optional latitude/longitude.	Storing the photo itself, multi-device sync, GIS map layers.
Notifications	Local daily reminder with WorkManager.	Server push alerts (e.g., disease outbreak warnings).

Cropora deliberately limits its treatment advice to cultural sanitation and resistant varieties, avoiding prescriptive chemical dosages due to regional pesticide regulations and advising users to consult local extension officers for severe pathogens.

1.5 Contributions

What this project contributes

- A working Android app with **online and offline diagnosis** that share one written and tested inference contract.
- A **reproducible model pipeline**: provenance and SHA-256 checks for the Keras file, conversion to TFLite, and an automated Keras–TFLite parity test.

- A **FastAPI backend** with input limits, explicit error codes, a health endpoint, a catalog endpoint and a Docker image.
- A **bilingual, source-linked guidance catalog** for all 38 classes and a **dual-mode user interface** with custom Canvas charts.
- An honest **verification record**, including the tests that fail or were not re-run (Chapter 5).

Model provenance

The neural network was **not trained in this project**. Cropora uses `model_4_mobilenet_finetuned.keras` from the open-source repository `Muhammad-Hassan12/Plant-Disease-Detector` (commit `f6165bd9`)[7]. Accuracy figures quoted by that repository were not re-measured here (see Section 6.2).

1.6 Organisation of the Report

Chapter 2 reviews related work and technologies. Chapter 3 describes the development process and the design of the model pipeline, backend and Android app. Chapter 4 presents the implemented system with real device screenshots and measured results. Chapter 5 reports the test suites and their outcomes. Chapter 6 concludes with limitations and future work.

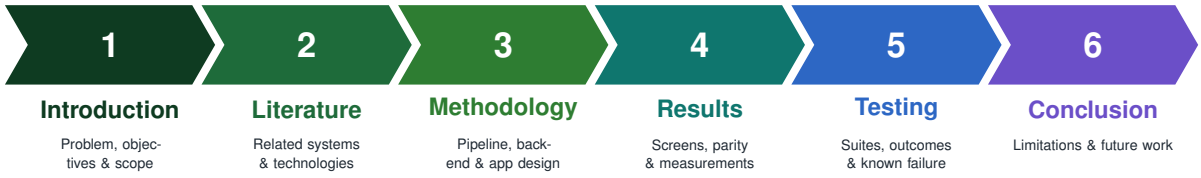


Figure 1.1: Roadmap of the report: from problem definition to verified results and future work.

CHAPTER 2

2 Research and Literature Review

IN THIS CHAPTER

2.1 Evolution of Plant Disease Diagnosis 5
2.2 Datasets: PlantVillage 5
2.3 Cloud versus On-Device Inference 6
2.4 Android, Kotlin and Jetpack 6
2.5 MobileNetV2, TensorFlow Lite and FastAPI 7
2.6 Related Systems 7

2.1 Evolution of Plant Disease Diagnosis

Plant disease diagnosis has progressed through three main stages. The first relied on visual scouting of lesions and discolouration by agronomists. This remains subjective and prone to confusion between pathogens with similar foliar symptoms—such as early blight (*Alternaria solani*) and Septoria leaf spot (*Septoria lycopersici*) on tomato. The second added laboratory serological and molecular assays (ELISA, PCR), which are highly specific for pathogens such as citrus greening (*Candidatus* Liberibacter spp.)[19] but require costly lab equipment and turnaround time.

The third stage leverages computer vision. Early approaches extracted hand-crafted colour, texture (GLCM) and shape descriptors for classical classifiers (SVMs, random forests), but were fragile to ambient lighting. Deep convolutional neural networks (CNNs) learn feature hierarchies directly from pixels, dramatically improving accuracy and robustness on leaf imagery[3, 4].

2.2 Datasets: PlantVillage

The benchmark dataset is **PlantVillage**, introduced by Hughes and Salathé[2]. Mohanty *et al.* trained CNNs on 54,306 images across 14 crops and 38 classes, reporting high laboratory accuracy while highlighting steep performance drops on images captured outside experimental settings[3]. Three characteristics are critical for Cropora:

- **Breadth.** The 38 classes span fungal, oomycete, bacterial, viral and mite pathologies alongside 12 healthy controls across 14 commercial crop species.

- **Label conventions.** Class identifiers follow folder conventions with triple underscores and historical spellings (e.g., `Orange___Haunglongbing_(Citrus_greening)`). Cropora preserves exact canonical strings in `labels-38.txt` and normalises display names in `DiseaseXmlParser`.
- **Domain gap.** PlantVillage leaves were photographed detached on uniform laboratory backdrops. Real-field photos introduce lighting, clutter and blur; Cropora therefore presents calibrated confidence and flags uncertain results rather than forcing diagnoses.

2.3 Cloud versus On-Device Inference

A fundamental design choice in mobile ML is the locus of execution:

> **Trade-off: server inference vs. on-device inference**
>
> **Server (FastAPI + Keras):** Allows zero-install model updates and centralized request logging, but mandates cellular connectivity and incurs image upload latency.
>
> **On-device (TensorFlow Lite):** Operates fully offline, guarantees user privacy by retaining photos locally, and avoids network latency, but requires compact model footprint and strict behavioral parity.

Modern mobile benchmarks confirm that mobile edge devices comfortably run lightweight CNN backbones[6]. However, dual-engine setups risk *training–serving skew*: different image decoders or scaling arithmetic can alter predictions silently. Cropora eliminates this divergence through an enforceable contract and automated parity tests (`model/parity_test.py`) requiring top-1 agreement and confidence delta ≤ 0.02.

2.4 Android, Kotlin and Jetpack

Cropora is developed as a native Android application in **Kotlin 1.9.22** to leverage low-level OS capabilities:

1. **Memory-mapped model execution.** `model.tflite` is stored uncompressed (`noCompress "tflite"`) and memory-mapped via `AssetFileDescriptor`, avoiding heap duplication.
2. **Asynchronous concurrency.** Inference runs on a dedicated single-thread executor, network calls use Retrofit coroutines/callbacks, and scans stream reactively via Room `Flow`.

3. **Room persistence.** Room provides compile-time verified SQLite schemas, entity mapping and robust schema migration paths.
4. **Localisation and scheduling.** WorkManager guarantees periodic 24-hour reminders, while `AppCompatDelegate.setApplicationLocales` manages in-app English/Bangla switching.

2.5 MobileNetV2, TensorFlow Lite and FastAPI

- **MobileNetV2**[5] employs depthwise separable convolutions and inverted residuals with linear bottlenecks. The backbone contains 2.31 M parameters (48,678 in the dense head).
- **TensorFlow Lite**[8] converts the Keras computational graph into a compact FlatBuffer, reducing disk size from 25.14 MB to 9.09 MB (−63.9%) without quantisation.
- **FastAPI**[9] provides an asynchronous ASGI backend with Pydantic validation, streaming multipart image uploads, and interactive OpenAPI documentation.

2.6 Related Systems

Commercial applications (Plantix, Agrio) provide broad coverage but rely on proprietary cloud backends and user accounts. Most academic PlantVillage prototypes end at exploratory notebooks. Table 2.1 compares these architectural archetypes against Cropora.

Table 2.1: Qualitative comparison of typical plant-diagnosis app types with Cropora

Feature	Cloud wrappers	Academic demos	Commercial apps	Cropora
Inference locus	Server only	Usually one engine	Mostly server	**User choice:** FastAPI server or on-device TFLite
Preprocessing check	Rare	Often missing	Not public	**Load-time checks** of shape, labels and rescaling
Low confidence	Rare	Top-1 only	Varies	**Uncertain flag**, user threshold 5–95%
Guidance	Generic	Label only	Proprietary	**38 entries** with extension source links
Bangla support	Rare	Rare	Varies	**Full UI and catalog** (expert review pending)
Interface modes	One	One	One	**Normal** and **Advanced**
History	Server account	None	Account	**Local Room database**, optional location

CHAPTER 3

3 Research Methodology

IN THIS CHAPTER

3.1	Development Methodology	8	3.4	Machine Learning Pipeline	10
3.2	Requirements	8	3.5	Backend Design (FastAPI)	13
3.3	System Architecture	9	3.6	Android Application Design	14

3.1 Development Methodology

Cropora was developed iteratively with a **contract-first** approach. Because the system spans three runtimes—a Python model pipeline, a FastAPI service and a Kotlin Android client—the shared rules were written down first and then enforced by tests in each component:

1. **Contract and provenance.** The 38-class label order, tensor shapes ($[1, 224, 224, 3] \rightarrow [1, 38]$), the in-graph rescaling rule and the SHA-256 hash of the source Keras file were fixed in `model/model_contract.py` and `backend-api/inference_contract.py`.
2. **Engines and guidance.** The FastAPI service, the TFLite conversion script and the 38-entry guidance catalog were built against that contract.
3. **Adaptive UI, persistence and localisation.** The Android app gained Normal and Advanced modes, chart views, the Room database and migration, optional location tagging, reminders and English/Bangla resources.
4. **Verification.** JVM unit tests, instrumented UI/database tests, model parity tests and backend API tests were written and run against the contract.

3.2 Requirements

Table 3.1: Main functional and non-functional requirements and where they are met

ID	Requirement	Implemented in
FR1	Capture or choose a leaf photo	`ScanActivity` (`TakePicture`, `GetContent`)
FR2	Diagnose online through a server	`RetrofitClient`, `POST /predict`
FR3	Diagnose offline on the phone	`TFLiteClassifier`
FR4	Flag low-confidence results	user threshold in `UiPreferences`
FR5	Show symptoms, treatment, prevention	`DiseaseRepository`, `diseases*.xml`
FR6	Save, view, share and delete scans	Room (`ScanDao`), `HistoryDetailActivity`
FR7	Browse the disease library	`DiseaseLibraryActivity`, `AdvancedLibraryFragment`
FR8	Summarise scan history	`ScanInsights`, chart views
FR9	Choose language and interface mode	`UiPreferences`, `ModeAwareActivity`
FR10	Daily scan reminder	`ScanReminderWorker`
NFR1	Same preprocessing on both engines	inference contract, parity test
NFR2	Work without internet	on-device engine, bundled catalog
NFR3	Reject bad input safely	HTTP 400/413/503, image-prepare error toast
NFR4	Reproducible, testable build	Gradle wrapper, unit tests, Dockerfile

3.3 System Architecture

Figure 3.1 shows the overall architecture. The Android client contains both inference paths. In online mode it sends the original image bytes to the FastAPI service, which preprocesses the image, runs the Keras model and returns a JSON object. In offline mode the app preprocesses the bitmap itself and runs the TFLite interpreter. Either way the result is shown on the result screen, can be saved to Room, and is enriched with guidance from the bundled XML catalog.

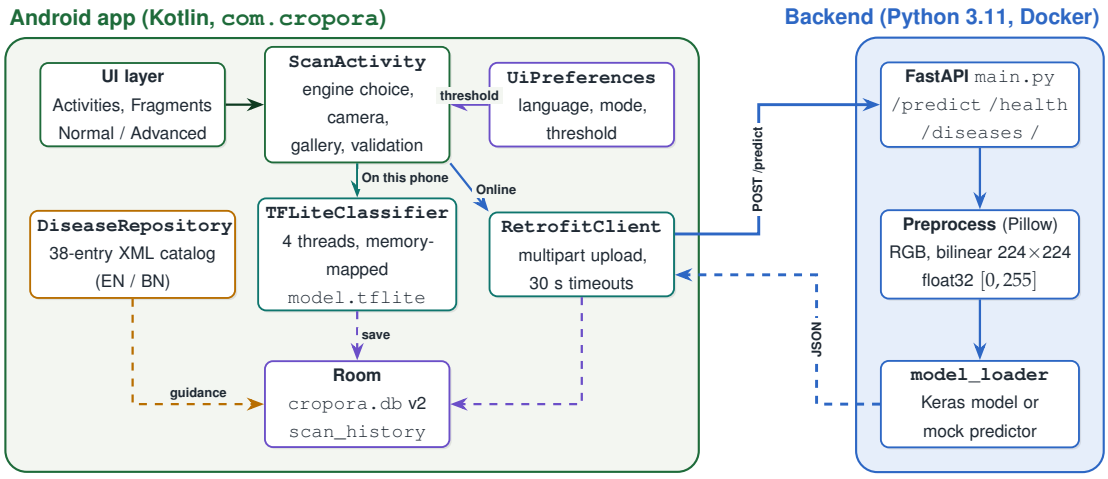


Figure 3.1: Cropora system architecture: an Android client with two inference paths and an optional FastAPI backend.

3.3.1 Dual-engine scan flow

Figure 3.2 shows what happens after the user taps *Check leaf*. The original design sketch of this flow is shown in Figure 3.3, and the implementation follows it.

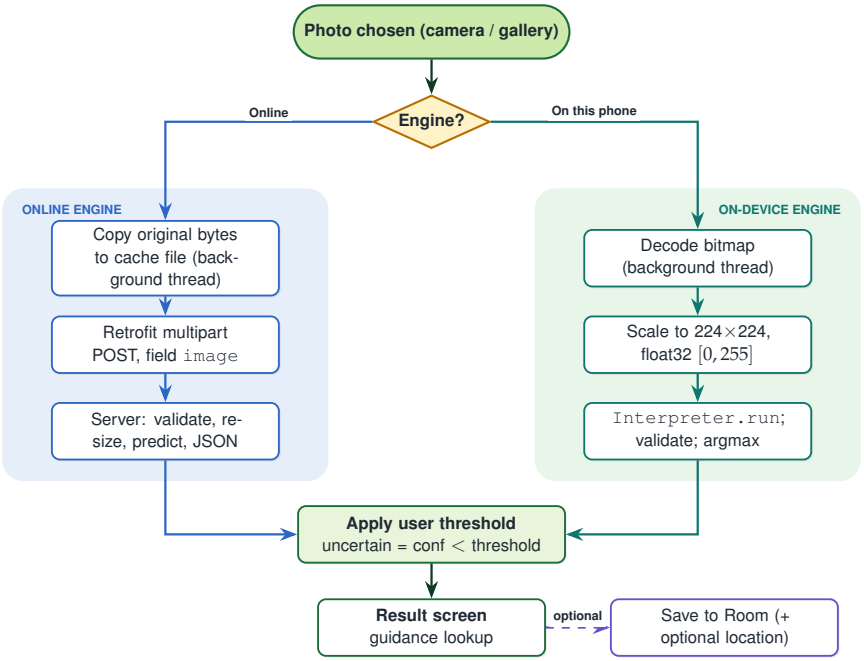


Figure 3.2: Dual-engine scan flow. In both branches the user's threshold decides the *uncertain* flag on the device.

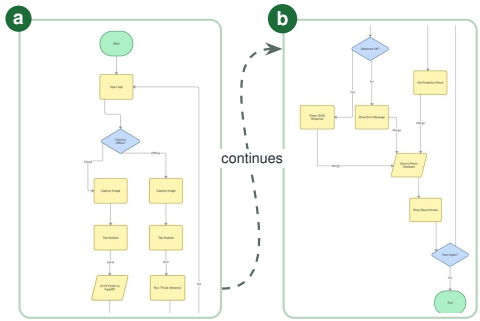


Figure 3.3: Original workflow sketch of the Cropora design, split into (a) upper and (b) lower halves for legibility. It shows the Cloud/Offline decision, the two paths merging, saving to Room and the result screen.

3.4 Machine Learning Pipeline

3.4.1 Model provenance

The deployed classifier is the file `model_4_mobilenet_finetuned.keras` from the open-source repository `Muhammad-Hassan12/Plant-Disease-Detector` (commit `f6165bd9`, MIT licence stated by the source)[7]. It is stored in this project as

`backend-api/models/cropora_model.keras`. The source author reports 98.75% accuracy; this was **not** independently measured in this project. `model/model_contract.py` records the file size and SHA-256 hash so that an accidentally replaced model is detected.

Table 3.2: Model artefacts (sizes and hashes measured on the files in the repository)

Artefact	Size (bytes)	SHA-256
Keras model (`.keras`)	25,143,175	`08f285aff6d9e1ab88d4d5b2269f1cc9 77714003755f8553887edbf8691b325f`
TFLite model (`.tflite`)	9,087,416	`494f60b2d0084c2dd804c5b9f289f815 7a5c1633d037df2cab054c4c49f8c0c0`

3.4.2 Network architecture

The model has eight layers and 2,306,662 parameters (Figure 3.4). Three augmentation layers (random flip, rotation, zoom) are inactive at inference time. The MobileNetV2 backbone (2,257,984 parameters) is followed by global average pooling, dropout and a 38-unit softmax layer (48,678 parameters). The $x/127.5 - 1$ rescaling is embedded in the graph, so callers must supply raw $[0, 255]$ values.

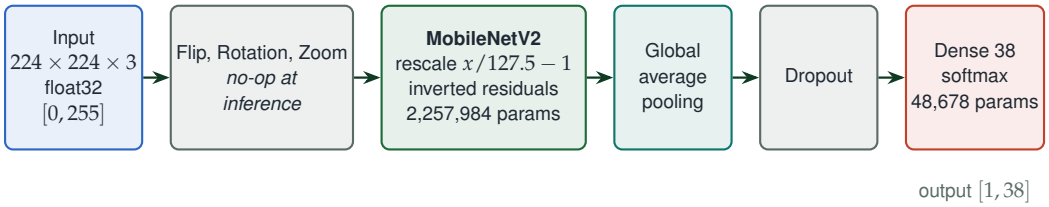


Figure 3.4: Layer structure of the deployed model (8 layers, 2,306,662 parameters).

3.4.3 The inference contract

The contract is the central design artefact of the project. It is stated once and checked in Python (`inference_contract.py`, `model_contract.py`) and Kotlin (`TFLiteClassifier`). Figure 3.5 and Table 3.3 summarise it.

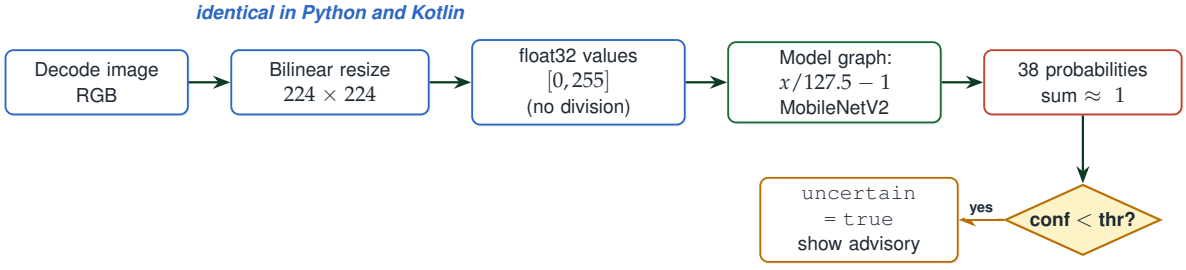


Figure 3.5: Preprocessing and post-processing contract shared by both inference engines.

Table 3.3: Inference contract rules and where each is enforced

Rule	Detail	Enforced by
Labels	Exactly 38 unique labels in the order of `labels-38.txt`	Backend `validate_labels`; Android `loadLabels` and label-order test
Input shape	$[1, 224, 224, 3]$, float32	`validate_shape`; classifier init
Rescaling	Embedded $x/127.5 - 1$ layer must exist; input stays in $[0, 255]$	`find_embedded_rescaling` (graph walk)
Output	Finite probabilities in $[0, 1]$ that sum to 1 (backend: rtol 10^{-5}, atol 10^{-6}; app: tolerance 1.1×10^{-5})	`validate_prediction`; `validateProbabilities`
Uncertainty	`uncertain = confidence < threshold`; default 0.50, adjustable 5–95% in Settings	backend flag; app re-applies the user's threshold

Android's `loadLabels` checks only label count and uniqueness at runtime; canonical ordering is checked by `AndroidLabelContractTest` in `model/test_tflite_contract.py`.

3.4.4 Conversion to TensorFlow Lite

`model/convert_model.py` converts the validated Keras model with TensorFlow 2.21.0 *without quantisation*, then `validate_tflite.py` checks the flatbuffer's tensor details and `parity_test.py` compares its outputs with Keras. The converted file is 9.09 MB, a 63.9% reduction from the 25.14 MB Keras file (Figure 3.6).

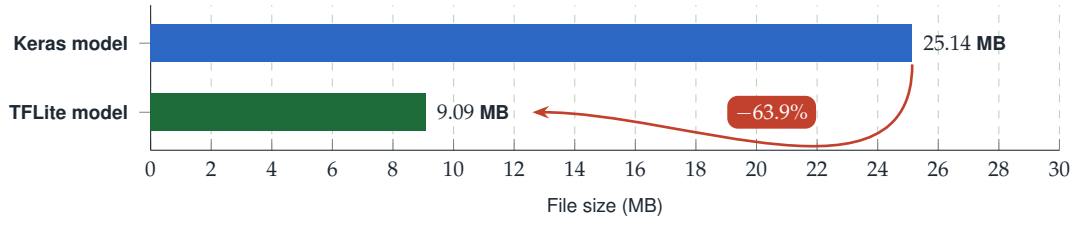


Figure 3.6: Model file size before and after TFLite conversion (25,143,175 B vs. 9,087,416 B; −63.9%).

The conversion pipeline validates FlatBuffer integrity and ensures contract fidelity. Following conversion, `validate_tflite.py` verifies the `float32` tensor signature ($[1, 224, 224, 3] \rightarrow [1, 38]$) and canonical label order before bundling `model.tflite` directly into application assets.

3.5 Backend Design (FastAPI)

The backend is a small FastAPI application (`backend-api/main.py`). Table 3.4 lists its four endpoints. A screenshot of the generated Swagger page is shown in Figure 3.7.

Table 3.4: Backend endpoints and response semantics

Method	Path	Response / Errors	Purpose & Semantics
GET	`/`	200	Root service health status confirmation.
GET	`/health`	200	Reports `status`, `use_mock`, `model_loaded`, paths and classes.
GET	`/diseases`	200	Returns full disease catalog loaded from `diseases.xml`.
POST	`/predict`	200, 400, 413, 503	Accepts multipart field `image`; returns 8-field prediction JSON.

Response format. `/predict` returns eight fields: `model_label`, `disease`, `confidence`, `uncertain`, `guidance_available`, `symptoms`, `treatment` and `prevention`.

Validation and errors. The handler verifies that the content type begins with `image/`, reads at most `MAX_IMAGE_SIZE_BYTES + 1` bytes (10 MB limit) in a single streaming call, and raises HTTP 400 if empty or un-decodable, or HTTP 413 if oversized. The raw bytes are decoded to RGB and bilinearly resized to 224×224 via Pillow without dividing by 255. When the real model is missing and mock mode is disabled, HTTP 503 is returned. Predictions are rigorously validated in `backend-api/inference_contract.py`: output arrays must match shape $(1, 38)$ of type `float32`, contain only finite probabilities bounded in $[0, 1]$, and sum to unity within tight numerical tolerances (rtol $= 10^{-5}$, atol $= 10^{-6}$).

Configuration and deployment. Settings come from environment variables (Table 3.5). The Docker image is based on `python:3.11-slim`, takes an `INSTALL_TENSORFLOW` build argument and defines a `HEALTHCHECK`. Pinned dependencies include FastAPI 0.109.0, Uvicorn 0.27.0, python-multipart 0.0.6, Pillow $\geq$11.0.0, NumPy $\geq$2.1.0, python-dotenv 1.0.0 and TensorFlow $\geq$2.21.0.

Table 3.5: Backend environment variables (`config.py`)

Variable	Default	Meaning
`USE_MOCK`	false	Force the mock predictor.
`MODEL_PATH`, `LABELS_PATH`	repository paths	Keras model and label file.
`IMAGE_SIZE`	224	Square input size.
`CONFIDENCE_THRESHOLD`	0.50	Server-side uncertainty threshold.
`MAX_IMAGE_SIZE_BYTES`	10 MB	Upload limit.
`ALLOWED_ORIGINS`	*	CORS origins.
`PORT`	8000	Listening port.

Mock predictor and fallback. For development without TensorFlow, a mock predictor maps the image's mean intensity to a class index and returns a fixed confidence (0.70, 0.78 or 0.86). In the current code, `load_predictor` also *falls back to the mock* (logging an error) when TensorFlow is missing, the model file is absent or validation fails. This is convenient for development but contradicts the README and one unit test; see Section 6.2.

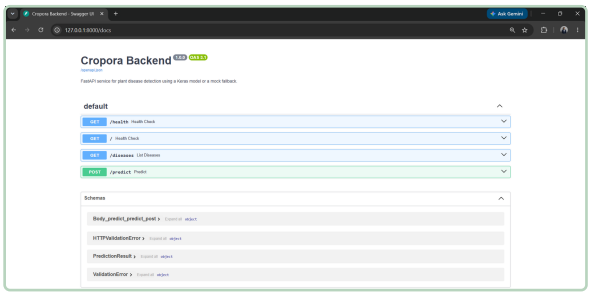


Figure 3.7: Swagger UI of the Cropora backend at `127.0.0.1:8000/docs`, listing `/health`, `/`, `/diseases` and `/predict`.

3.6 Android Application Design

3.6.1 Build configuration

The app uses Kotlin 1.9.22, Android Gradle Plugin 8.5.2, Gradle 8.10 and JDK 17, with `compileSdk` and `targetSdk` 34, `minSdk` 24, namespace `com.cropora` and `versionName` 0.1.0. The backend URL is a build-time default (`BuildConfig.API_BASE_URL = http://10.0.2.2:8000/`, overridable with `-PCROPORA_API_BASE_URL`) that the user can change at run time in Settings. Table 3.6 lists the libraries.

Table 3.6: Main dependencies declared in `app/build.gradle`

Area	Libraries (version)
UI	core-ktx 1.12.0, appcompat 1.6.1, Material 1.11.0, ConstraintLayout 2.1.4, RecyclerView 1.3.2, lifecycle 2.7.0
Networking	Retrofit 2.9.0 with Gson converter, OkHttp logging-interceptor 4.12.0
On-device ML	TensorFlow Lite 2.14.0
Persistence & jobs	Room 2.6.1 (kapt), WorkManager 2.9.0
Testing	JUnit 4.13.2, Espresso 3.5.1, Gson 2.10.1

3.6.2 Screens and navigation

The manifest declares nine activities: `MainActivity` (launcher), `ScanActivity`, `ResultActivity`, `HistoryActivity`, `HistoryDetailActivity`, `DiseaseLibraryActivity`, `DiseaseDetailActivity`, `SettingsActivity` and `AnalyticsActivity`. Rather than using a separate activity, Advanced mode inflates `activity_main_advanced.xml` within `MainActivity` and switches four fragments—`AdvancedHomeFragment`, `AdvancedScansFragment`, `AdvancedLibraryFragment` and `AdvancedInsightsFragment`—via `AdvancedNavigator`. Screens requiring mode-specific styling extend `ModeAwareActivity`, which applies `Theme_Cropora_Advanced` and recreates the activity upon mode switches (Figure 3.8).

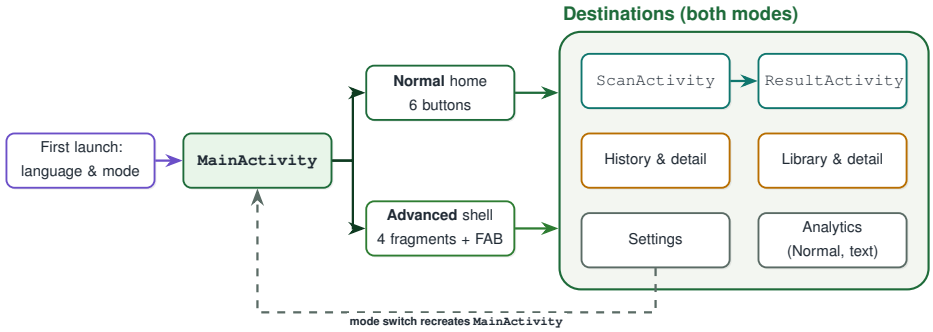


Figure 3.8: Simplified navigation map. Mode switching in Settings recreates `MainActivity` with the other layout.

3.6.3 On-device classifier

`TFLiteClassifier` memory-maps `model.tflite` from assets, running on four threads and validating input shape $[1, 224, 224, 3]$ (`float32`) against 38 labels. `Bitmap.createScaledBitmap` resizes images to 224×224, loading 602,112 bytes ($224 \times 224 \times 3 \times 4$) of raw RGB values into a direct buffer. After inference, output probabilities sum to 1 within 1.1×10^{-5}, the top class is resolved via arg-max, and the user's uncertainty threshold is applied, defaulting safely if no catalog entry matches.

3.6.4 Online path

For the online engine, `ScanActivity` copies the original image bytes to a cache file on a background executor (no re-compression), builds a multipart body with field name `image` and enqueues the Retrofit call. The Retrofit client is cached per base URL, uses 30-second OkHttp timeouts, and takes its URL from `ServerPreferences`, which accepts only `http` and `https` URLs. Because the server uses its own threshold, the app re-computes `uncertain` with the user's threshold when the response arrives.

3.6.5 Local database

Scans are stored in Room database `cropora.db`, version 2, with exported schemas (`1.json`, `2.json`). `MIGRATION_1_2` adds the nullable `latitude` and `longitude` columns. The *photo itself is not stored*; the entity keeps the prediction and guidance text (Figure 3.9). `ScanDao` provides `insertScan` (ignore on conflict), `observeAllScans` (a `Flow` ordered by newest first), `getScanById` and `deleteScanById`.

scan_history — entity `ScanRecord` • `cropora.db` v2

KEY	COLUMN	TYPE	NOTES
PK	id	INTEGER	autoincrement
UQ	result_id	TEXT	unique index (UUID)
	model_label	TEXT	e.g. `Tomato___Early_blight`
	disease	TEXT	display name
	confidence	REAL	0…1
	uncertain	INTEGER	boolean
	guidance_available	INTEGER	boolean
	symptoms / treatment / prevention	TEXT	guidance snapshot
v2	latitude, longitude	REAL	nullable (added in v2)
	timestamp	INTEGER	epoch milliseconds

Figure 3.9: Schema of the single Room table, as defined in `ScanRecord.kt`.

Result, history and insights: `ResultActivity` displays diagnoses and three guidance sections, offering sharing and optional location tagging. `HistoryDetailActivity` handles `geo:` map intents and scan deletion. `ScanInsights` computes totals, healthy/diseased counts, location tagging rates, and 7-day activity series. In Advanced mode, scans are prioritised into *needs review* (uncertain), *healthy*, or *diseased*, matching the donut chart metrics (2 healthy, 14 diseased, 0 review).

Custom chart views: `DonutChartView` renders segmented Canvas arcs with rounded caps; `BarChartView` draws dynamic 7-day activity bars with current-day highlighting. Normal mode presents statistics as plain text in `AnalyticsActivity`.

Guidance catalog: The catalog bundles `diseases.xml` (English) and `diseases-bn.xml` (Bangla) in assets with 38 entries containing symptoms, treatment, prevention and source URLs (Appendix B).

> **Status of the guidance content**
>
> Guidance entries represent an **AI-authored synthesis** of cited extension sources rather than formal agronomist certification; Bangla translations require expert agronomic validation before field deployment. The catalog avoids specific pesticide doses and directs growers to local authorities.

Localisation and preferences: Preferences are stored in `SharedPreferences` (`cropora_settings`): `app_language` (`en`/`bn`), `setup_complete`, `advanced_ui_enabled` and `confidence_threshold` (0.50). Languages are managed via `AppCompatDelegate.setApplicationLocales`. String resources include 132 English and 129 Bangla entries in `strings.xml`, 82 in `advanced_strings.xml`, and dual-locale disease names.

Reminders and permissions: A `PeriodicWorkRequest` runs every 24 hours under `cropora_daily_scan_reminder` (`UPDATE` policy), posting notifications in channel `cropora_scan_reminders`. Permissions include `CAMERA`, coarse/fine location, and `POST_NOTIFICATIONS` (Android 13+).

Visual identity: Both modes draw colours from application resources (Figure 3.10). Normal mode uses classic leaf greens; Advanced mode uses dark emerald with lime accents and semantic status tokens.

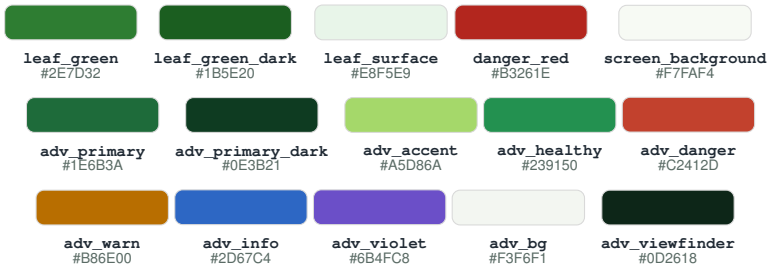


Figure 3.10: Colour tokens from `res/values/colors.xml`: Normal mode (top row) and Advanced mode (middle and bottom rows).

CHAPTER 4

4 Results and Discussion

IN THIS CHAPTER

4.1 Implementation at a Glance 18
4.2 Normal Mode 18
4.3 Advanced Mode 19
4.4 Saved Scans from the Test Phone 21
4.5 Keras versus TFLite Parity 22
4.6 Inference Latency 22
4.7 Backend Verification Evidence 22
4.8 Disease Taxonomy 23

This chapter presents the implemented application using screenshots captured on a physical Android phone (720×1600 pixels; the status and navigation bars were cropped), followed by measured verification results. All screenshots are unedited captures of the running app.

4.1 Implementation at a Glance

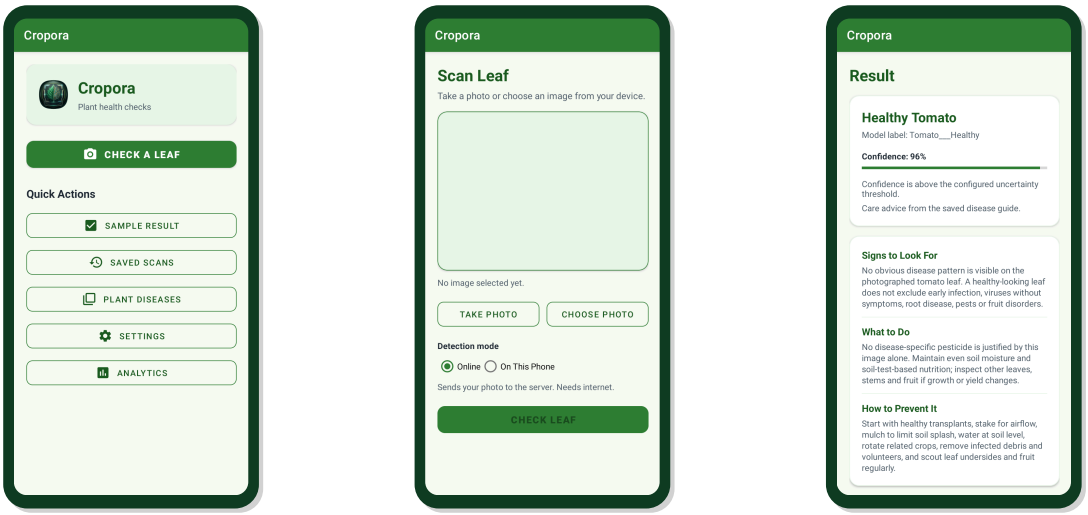
Table 4.1: Implementation summary (counts taken from the repository)

Item	Value
Activities / Advanced fragments	9 / 4 (plus `AdvancedNavigator`, `AdvancedUi`)
Custom views	`DonutChartView`, `BarChartView`
Disease classes and crops	38 classes (26 disease, 12 healthy), 14 crops
Guidance entries	38 English + 38 Bangla (app assets) + 38 (backend copy)
String resources	132 EN / 129 BN (main), 82 / 82 (Advanced)
Database	Room v2, one table, migration 1→2
Tests	12 JVM unit, 24 instrumented, 21 backend, 12 model-pipeline
Debug APK (measured build)	33,082,916 bytes (about 31.6 MiB)

4.2 Normal Mode

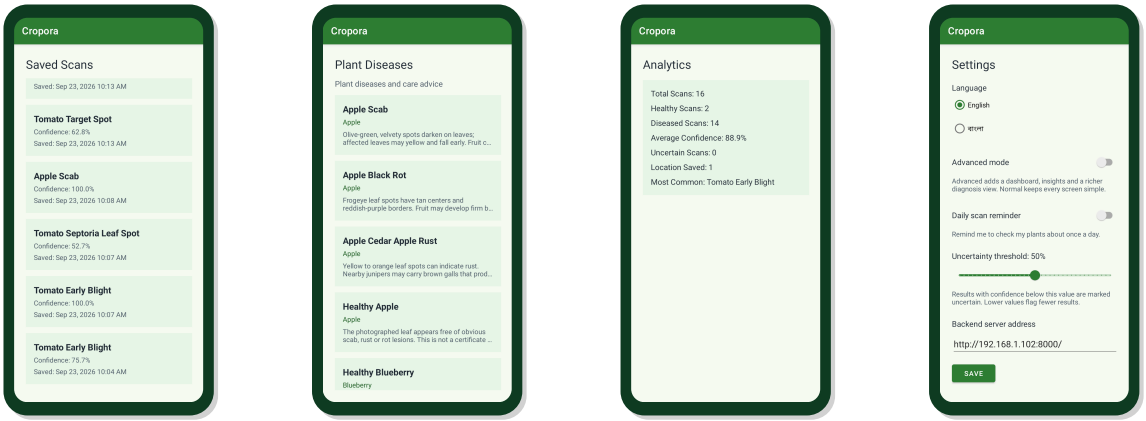
Normal mode keeps a linear, button-driven flow with large controls and plain language (Figures 4.1 and 4.2). The home screen has one prominent *Check a leaf* action and five quick actions. The scan screen lets the user take or choose a photo and select the *detection mode*: *Online* (server) or *On this phone* (offline). The result screen shows the

diagnosis name, the model label, a confidence bar, the uncertainty message and three guidance sections.



(a) Home **(b) Scan: photo and engine** **(c) Result (built-in sample)**

Figure 4.1: Normal mode, main flow. The result in (c) is the app's built-in *Sample result* (Healthy Tomato, `Tomato___Healthy`, 96%) with the sections *Signs to Look For*, *What to Do* and *How to Prevent It*.

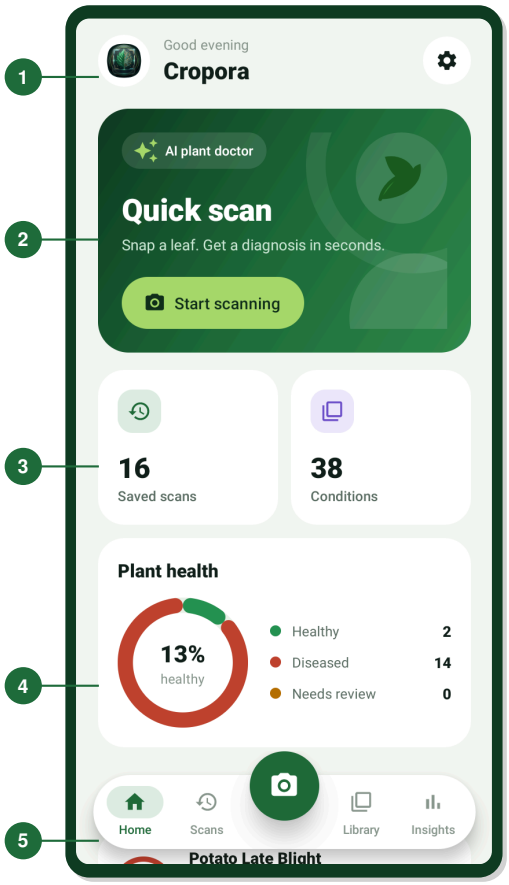


(a) Saved scans **(b) Plant diseases** **(c) Analytics** **(d) Settings**

Figure 4.2: Normal mode, supporting screens. (c) is a text summary: 16 scans, 2 healthy, 14 diseased, average confidence 88.9%, 0 uncertain, 1 scan with location, most common result Tomato Early Blight. (d) offers English/Bangla, the Advanced-mode switch, the daily reminder, the uncertainty threshold slider (50%) and the backend URL.

4.3 Advanced Mode

Advanced mode replaces the button list with a dashboard and a floating bottom navigation bar (*Home, Scans, camera button, Library, Insights*). Figure 4.3 annotates the home screen, and Figures 4.4 and 4.5 show the other screens.



1 **Header:** greeting, logo, settings shortcut.

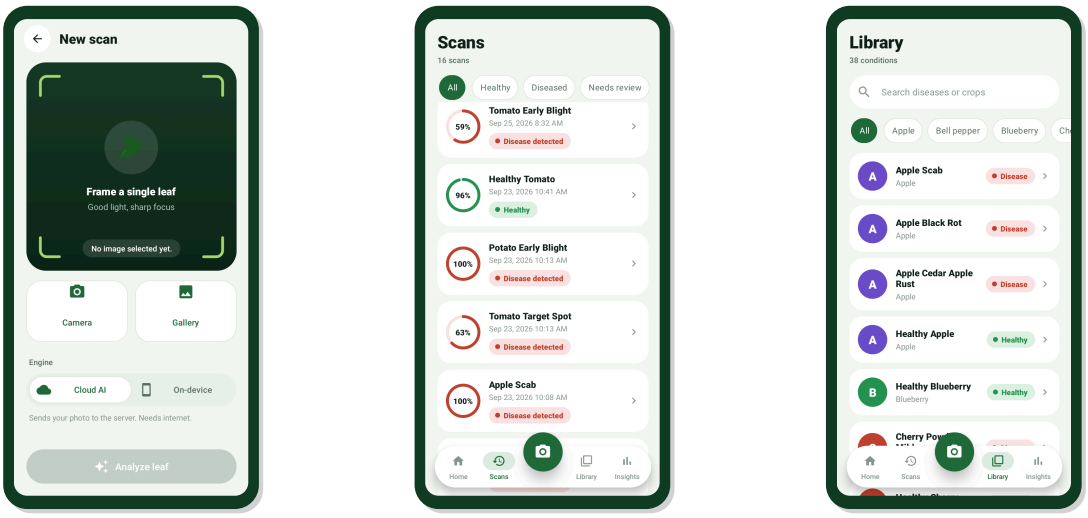
2 **Quick-scan hero:** one tap to start a scan.

3 **Counters:** saved scans (16) and catalog size (38 conditions).

4 **Plant-health donut:** 13% healthy; legend 2 healthy, 14 diseased, 0 needs review (custom `DonutChartView`).

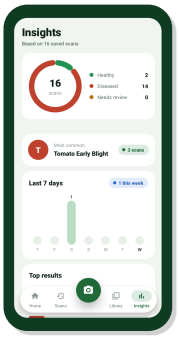
5 **Floating navigation bar** with a central camera button.

Figure 4.3: Annotated Advanced-mode home screen.

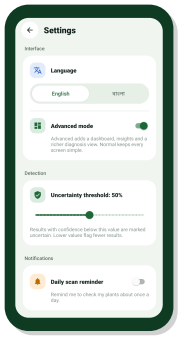


(a) New scan (b) Scans (c) Library

Figure 4.4: Advanced mode (1/2). (a) Viewfinder-style scan page with Camera and Gallery tiles and an *Engine* control (Cloud AI / On-device). (b) Scan history with filter chips (All, Healthy, Diseased, Needs review), confidence rings and status pills. (c) Searchable library of 38 conditions with crop chips and Disease/Healthy badges.



(a) Insights: health donut and activity chart



(b) Settings: detection and interface options

Figure 4.5: Advanced mode supporting screens. (a) Insights: health donut (16 scans), most common result (Tomato Early Blight), last-seven-days bar chart and top results. (b) Settings grouped into Interface, Detection and Notifications cards.

4.3.1 Normal versus Advanced mode

Table 4.2: Comparison of the two presentation modes

Aspect	Normal mode	Advanced mode
Navigation	One primary scan button and five quick actions; each opens its own screen	Floating bottom bar with four tabs and a camera button
Engine selector	Radio buttons: *Online* / *On This Phone*	Segmented control: *Cloud AI* / *On-device*
History	Plain list of saved scans	Filter chips, confidence rings, status pills
Statistics	Text summary (`AnalyticsActivity`)	Donut and bar charts (Home, Insights)
Library	List with symptom snippets	Search, crop chips, avatar letters, badges
Theme	`leaf_green` palette	`adv_*` palette, rounded cards

4.4 Saved Scans from the Test Phone

The saved-scan screens of the test phone show real results, for example those in Table 4.3. Their confidence values (such as 62.8% and 52.7%) are not values the mock predictor can produce (it returns only 0.70, 0.78 or 0.86), so they come from real model inference. The database does not record which engine produced a scan, so the engine used for each entry cannot be stated.

Table 4.3: Selected entries visible in the scan history of the test phone (Advanced Scans and Normal Saved Scans screens)

Result	Confidence	Status	Date
Tomato Early Blight	59%	Disease	25 Sep 2026
Healthy Tomato	96%	Healthy	23 Sep 2026
Potato Early Blight	100%	Disease	23 Sep 2026
Tomato Target Spot	63% (62.8%)	Disease	23 Sep 2026
Apple Scab	100%	Disease	23 Sep 2026

These entries demonstrate that the scan–result–save–history chain works on a device. They are an *operating demonstration*, not an accuracy evaluation: the true condition of each photographed leaf was not recorded, so no correctness rate can be computed from them.

4.5 Keras versus TFLite Parity

To confirm that the converted model behaves like the original, the project's own test (`model/test_tflite_contract.py`) feeds identical images to the Keras model and to the TFLite interpreter and compares the outputs. It was re-run for this report on a desktop machine (TensorFlow 2.21.0, Keras 3.15.1). The fixtures are three synthetic gradient images (96×128 pixels), so the test checks *numerical equivalence of the two formats*, not accuracy on real leaves.

Table 4.4: Measured Keras–TFLite parity on three synthetic fixtures (tolerance 0.02)

Fixture	Top-1 label (both engines)	Keras	TFLite	$\lvert\Delta\rvert$
0	`Corn___Healthy`	0.940499	0.940500	5.96×10^{-7}
1	`Blueberry___Healthy`	0.995868	0.995868	1.19×10^{-7}
2	`Blueberry___Healthy`	0.999561	0.999561	0

Top-1 agreement was 3 of 3, and the largest confidence difference was about 6×10^{-7}—more than four orders of magnitude below the 0.02 tolerance. The test confirms that the conversion preserved the model's behaviour on these inputs. It does not test real photographs, and it was run on a desktop CPU, not on an Android device.

4.6 Inference Latency

Latency was measured on the development PC (x86 CPU with AVX2/AVX_VNNI), not on a phone, using the same Keras and TFLite files. The TFLite interpreter used four threads, as in the app.

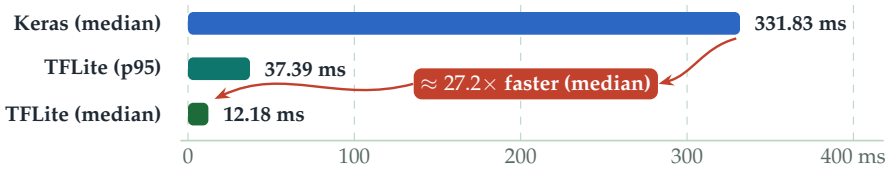


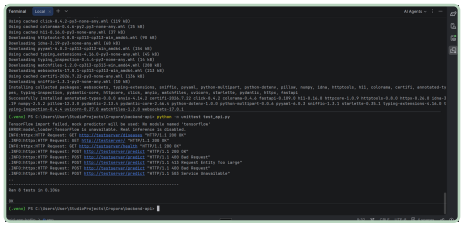
Figure 4.6: Desktop CPU inference time: TFLite (4 threads) median 12.18 ms and 95th percentile 37.39 ms; Keras median 331.83 ms.

On this desktop, TFLite was roughly 27 times faster than Keras at the median. **These numbers do not describe phone performance.** No on-phone latency benchmark was recorded for this report; the instrumented `OfflineInferencePerformanceTest` only checks a loose smoke budget of 15,000 ms and is not a performance measurement.

4.7 Backend Verification Evidence

A terminal capture of the backend test run and the health endpoint is shown in Figure 4.7. At that time TensorFlow was not installed in the virtual environment, so the

service ran with the mock predictor: the health response shows `"use_mock":true` and `"model_loaded":false`. The eight API tests passed in 0.106 s, covering `/diseases`, `/`, `/health` and the `/predict` responses 200, 400, 413 and 503.



(a) API test run: 8 tests passing in 0.106 s.



(b) Health endpoint response (`/health`).

Figure 4.7: Backend verification evidence: (a) automated test execution and (b) mock-mode service health status.

4.8 Disease Taxonomy

Table 4.5 lists how the 38 classes are distributed across the 14 crops (26 disease classes and 12 healthy classes).

Table 4.5: Classes per crop (from `labels-38.txt`)

Crop	Classes	Conditions
Tomato	10	bacterial spot, early blight, late blight, leaf mold, Septoria leaf spot, spider mites, target spot, yellow leaf curl virus, mosaic virus, healthy
Apple	4	scab, black rot, cedar apple rust, healthy
Corn	4	gray leaf spot, common rust, northern leaf blight, healthy
Grape	4	black rot, Esca, leaf blight, healthy
Potato	3	early blight, late blight, healthy
Cherry	2	powdery mildew, healthy
Peach	2	bacterial spot, healthy
Bell pepper	2	bacterial spot, healthy
Strawberry	2	leaf scorch, healthy
Orange	1	Huanglongbing (citrus greening)
Squash	1	powdery mildew
Blueberry, Raspberry, Soybean	1 each	healthy
Total	**38**	26 disease conditions + 12 healthy

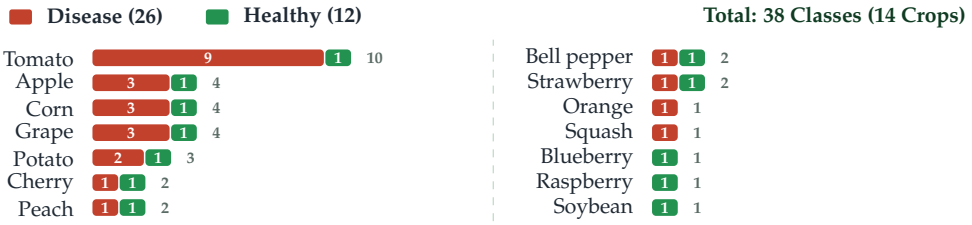


Figure 4.8: Disease and healthy classes per crop (26 disease + 12 healthy = 38).

CHAPTER 5

5 Testing and Verification

IN THIS CHAPTER

5.1 Test Strategy and Environment 24
5.2 Unit and Pipeline Verification 24
5.3 Backend Tests and Known Issue 25
5.4 Instrumented and Manual Verification 25

5.1 Test Strategy and Environment

Verification follows the architectural layers of Cropora. Fast, deterministic tests run on the development machine to validate contracts, labels, tensor shapes, probability bounds, parsing, insights statistics and localisation. Slower instrumented tests target UI navigation, database migrations and background workers on Android. Automated suites were executed on 22 October 2026 on the development PC using standard `unittest` (model-pipeline in TensorFlow 2.21 / Keras 3.15; backend in FastAPI 0.123 / NumPy 2.3) and Gradle JDK 17 for Android. Table 5.1 summarizes the suites and execution results.

Table 5.1: Test suites and results (executed 22 October 2026)

Suite	Scope and Focus	Tests	Result
Model pipeline (`model/`)	Contract, label file, TFLite flatbuffer, Keras–TFLite parity	12	12 pass
Backend (`backend-api/`)	Endpoints, HTTP error codes, contract, loader modes	21	19 pass, 1 skip, 1 fail
Android JVM unit	Catalog integrity, localisation, response parsing, insights	12	12 pass
Android instrumented	UI modes, analytics, migration 1→2, DAO, language, settings	24	not re-run

5.2 Unit and Pipeline Verification

Model Pipeline (12/12 passed): Tests in `model/` validated canonical 38-class ordering, verified file hashes and dimensions, confirmed output shape $[1, 38]$, and proved numerical parity between Keras and TFLite ($|\Delta| < 6 \times 10^{-7}$, Section 4.5).

Android JVM Tests (12/12 passed): Verified catalog completeness, placeholder integrity in Bangla, 8-field response extraction, confidence formatting, and `ScanInsights` date aggregation. The Gradle build completed cleanly, yielding a 31.6 MiB APK.

5.3 Backend Tests and Known Issue

Of 21 defined backend tests, 19 passed, one was skipped (`EmbeddedRescalingGraphTest`), and one failed with six sub-cases in `test_model_loader`.

Known failing test: loader fallback

`test_model_loader.LoadPredictorTest` fails across six sub-cases (*no_tensorflow, missing_file, wrong_size, load_error, invalid_model, invalid_rescaling*). The test expects the loader to set `use_mock = False` on failure, but the code falls back to the mock predictor and sets `use_mock = True`.

Impact & Resolution: A damaged model answers with mock predictions instead of HTTP 503. Deployments must check `/health` for `model_loaded = true`. The loader should fail strictly in future releases.

5.4 Instrumented and Manual Verification

The repository defines 24 instrumented tests in `app/src/androidTest` across eight functional test classes:

- **Navigation & UI (14 tests):** `AdvancedModeInstrumentedTest` (6), `ResultActivityInstrumentedTest` (5), and `SettingsActivityInstrumentedTest` (3).
- **Data & Database (4 tests):** `ScanDaoInstrumentedTest` (1), `AppDatabaseMigrationTest` (1), and `AnalyticsActivityInstrumentedTest` (2).
- **System & Runtime (6 tests):** `LanguageInstrumentedTest` (5) and `OfflineInferencePerformanceTest` (1).

These instrumented tests require an Android emulator or connected device and were not re-run for this report. Manual checks on a physical phone verified end-to-end operation across all workflows (Chapter 4).

CHAPTER 6

6 Conclusion and Future Work

IN THIS CHAPTER

6.1 Summary 26
6.2 Limitations and Known Issues 26
6.3 Future Work 27
6.4 Final Remarks 27

6.1 Summary

Cropora classifies leaf photographs into 38 PlantVillage classes across 14 crops using an online FastAPI backend or an offline TensorFlow Lite model, accompanied by bilingual guidance in English and Bangla. Its core engineering foundation is a strict, validated inference contract enforced across Python and Kotlin. Table 6.1 summarizes project objectives and achieved outcomes.

Table 6.1: Objectives and outcomes

ID	Outcome	Evidence
O1	Online and on-device engines, selectable on scan screen	Figures 4.1, 4.4; Table 4.2
O2	Contract enforced across Python and Kotlin	Table 3.3; Section 3.6.3
O3	TFLite conversion with verified parity (3/3 agreement, $\|\Delta\| < 6 \times 10^{-7}$)	Table 4.4
O4	38 bilingual guidance entries with extension source links	Chapter 3; Appendix B
O5	Dual-mode interface (Normal and Advanced) with custom charts	Figures 4.3, 4.5
O6	English and Bangla support via per-app locales	Figure 4.2(d); Chapter 3
O7	Room SQLite database, migration 1→2, location, share, delete	Figure 3.9; Table 4.3
O8	Automated verification across backend, pipeline and Android	Table 5.1

6.2 Limitations and Known Issues

The following limitations contextualize the findings of this report:

1. **Third-party classifier:** Model weights from `Muhammad-Hassan12/Plant-Disease-Detector`[7]; 98.75% accuracy was not independently verified on real-field photos[3].

2. **Synthetic parity checks:** Parity tests confirm numerical equivalence on synthetic gradients; field testing remains future work.
3. **Version drift:** Source model TensorFlow 2.19.1 / Keras 3.10.0, conversion with TensorFlow 2.21.0, and Android runtime TFLite 2.14.0.
4. **Loader fallback inconsistency:** One backend test fails because `load_predictor` defaults to the mock predictor rather than HTTP 503 when the real model fails.
5. **Instrumented tests unexecuted:** 24 device tests exist but were not re-run for this report.
6. **Desktop latency numbers:** Benchmark timings reflect a PC CPU rather than mobile hardware.
7. **AI-synthesised guidance:** Clinical entries synthesise extension fact sheets; formal agronomist review (especially for Bangla) is required before production deployment.
8. **Scope boundary:** Only the 38 trained classes are recognized; arbitrary non-leaf inputs are guarded only by the confidence threshold.

6.3 Future Work

- Resolve loader inconsistency by rejecting failed models with explicit HTTP 503 responses.
- Unify library versions across TensorFlow, Keras and TFLite; validate parity on physical phones.
- Benchmark accuracy on an independent Bangladeshi crop dataset; fine-tune weights if needed.
- Implement out-of-distribution (non-leaf) rejection, INT8 quantisation, and NNAPI delegates.
- Integrate instrumented tests into continuous integration and profile real on-device execution times.
- Conduct agronomic expert review of bilingual guidance texts and expand the species taxonomy.

6.4 Final Remarks

Cropora demonstrates that a pre-trained neural network can become an accessible, resilient agricultural tool when systemic contracts, numerical parity, uncertainty gating, bilingual guidance, and disciplined tests are prioritized alongside model architecture.

References

[1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, "The global burden of pathogens and pests on major food crops," *Nature Ecology & Evolution*, vol. 3, pp. 430–439, 2019.

[2] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[3] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, art. 1419, 2016.

[4] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[6] A. Ignatov *et al.*, "AI Benchmark: All about deep learning on smartphones in 2019," in *IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, 2019, pp. 3617–3635.

[7] M. Hassan, "Plant-Disease-Detector (model_4_mobilenet_finetuned.keras)," GitHub repository, commit f6165bd9. [Online]. Available: https://github.com/Muhammad-Hassan12/Plant-Disease-Detector

[8] Google, "TensorFlow Lite guide." [Online]. Available: https://www.tensorflow.org/lite

[9] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[10] University of Minnesota Extension, "Late blight." [Online]. Available: https://extension.umn.edu/disease-management/late-blight

[11] University of Minnesota Extension, “Apple scab.” [Online]. Available: https://extension.umn.edu/plant-diseases/apple-scab

[12] Clemson Cooperative Extension, Home & Garden Information Center, “Tomato diseases and disorders.” [Online]. Available: https://hgic.clemson.edu/factsheet/tomato-diseases-disorders/

[13] Clemson Cooperative Extension, Home & Garden Information Center, “Peach diseases.” [Online]. Available: https://hgic.clemson.edu/factsheet/peach-diseases/

[14] NC State Extension, “Bacterial spot of pepper and tomato.” [Online]. Available: https://content.ces.ncsu.edu/bacterial-spot-of-pepper-and-tomato

[15] UC Statewide IPM Program, “Grape: Esca (black measles).” [Online]. Available: https://ipm.ucanr.edu/PMG/r302100511.html

[16] UF/IFAS Extension, “Target spot of tomato (PP351).” [Online]. Available: https://ask.ifas.ufl.edu/publication/PP351

[17] Pacific Northwest Plant Disease Management Handbook, “Cherry powdery mildew.” [Online]. Available: https://pnwhandbooks.org/plantdisease/host-disease/cherry-prunus-spp-powdery-mildew

[18] University of Kentucky, “Leaf spots of grapes.” [Online]. Available: https://fruitscout.mgcafe.uky.edu/leaf-spots-grapes

[19] USDA APHIS, “Citrus greening and Asian citrus psyllid.” [Online]. Available: https://www.aphis.usda.gov/plant-pests-diseases/citrus-diseases/citrus-greening-and-asian-citrus-psyllid

APPENDIX A

Repository Structure and Reproduction

Table A.1: Main folders of the repository

Path	Content
`android-app-kotlin/`	Android app (Gradle project), unit and instrumented tests, Room schemas
`backend-api/`	FastAPI service, inference contract, model loader, labels, catalog, Dockerfile, tests
`model/`	Contract, inspection, conversion (`convert_model.py`), TFLite validation, parity tests
`docs/`	Branding, development evidence, `guidance-sources.md`, Kotlin notes

Verification results can be reproduced by executing the automated test suites:

- **Backend API:** Run `python -m unittest discover` within `backend-api/`.
- **Model Pipeline:** Run `python -m unittest discover` within `model/` (requires TensorFlow 2.21+).
- **Android Unit Tests & Build:** Execute `./gradlew testDebugUnitTest assembleDebug` within `android-app-kotlin/` (requires JDK 17).

APPENDIX B

Guidance Sources

Each guidance entry carries its source URL(s) in the XML catalog; the full per-class table is in `docs/guidance-sources.md`. The main sources are the University of Minnesota Extension[10, 11], Clemson HGIC[12, 13], NC State Extension[14], UC IPM[15], UF/IFAS[16], the Pacific Northwest Handbooks[17], the University of Kentucky[18] and USDA APHIS[19]. As stated in the repository, the entries are an AI-authored synthesis of these sources and not an endorsement by them.

Table B.1: Primary institutional agricultural extension sources

Institution	Focus Crops	Addressed Conditions
Univ. of Minnesota Extension[10, 11]	Potato, Tomato, Apple	Late blight, Apple scab
Clemson HGIC[12, 13]	Tomato, Peach	Bacterial spot, foliar diseases
NC State Extension[14]	Bell Pepper, Tomato	Bacterial spot
UC IPM[15]	Grape	Esca (Black Measles)
UF/IFAS Extension[16]	Tomato	Target spot, fungal blights
PNW Plant Disease Handbooks[17]	Cherry, Stone fruits	Powdery mildew
Univ. of Kentucky Extension[18]	Grape	Leaf blight (Isariopsis), black rot
USDA APHIS[19]	Citrus / Orange	Huanglongbing (Citrus greening)

APPENDIX C

The 38 Model Classes

The tables list the labels in the exact order of `labels-38.txt`. The badge is the model output index (0–37); each full label is *Crop*___*Condition*. Badge colour shows the class type: Disease (26 classes) and Healthy (12 classes).

#	Crop	Condition (model label suffix)
0	Apple	Apple_scab
1	Apple	Black_rot
2	Apple	Cedar_apple_rust
3	Apple	Healthy
4	Blueberry	Healthy
5	Cherry	Powdery_mildew
6	Cherry	Healthy
7	Corn	Cercospora Gray_leaf_spot
8	Corn	Common_rust
9	Corn	Northern_Leaf_Blight
10	Corn	Healthy
11	Grape	Black_rot
12	Grape	Esca_(Black_Measles)
13	Grape	Leaf_blight_(Isariopsis)
14	Grape	Healthy
15	Orange	Haunglongbing_(Citrus_ greening)
16	Peach	Bacterial_spot
17	Peach	Healthy
18	Pepper,_bell	Bacterial_spot
19	Pepper,_bell	Healthy
20	Potato	Early_blight
21	Potato	Late_blight
22	Potato	Healthy
23	Raspberry	Healthy
24	Soybean	Healthy
25	Squash	Powdery_mildew
26	Strawberry	Leaf_scorch
27	Strawberry	Healthy
28	Tomato	Bacterial_spot
29	Tomato	Early_blight
30	Tomato	Late_blight
31	Tomato	Leaf_Mold
32	Tomato	Septoria_leaf_spot
33	Tomato	Spider_mites Two-spotted
34	Tomato	Target_Spot
35	Tomato	Tomato_Yellow_Leaf_Curl
36	Tomato	Tomato_mosaic_virus
37	Tomato	Healthy



Cropora

Scan a leaf. Know the disease. Act early.

Hybrid cloud and on-device plant disease diagnosis for Android, with English–Bangla guidance for 38 classes across 14 crops.

Online FastAPI engine **Offline TFLite engine** **EN / BN guidance**

github.com/M-F-Tushar/Cropora

Department of Computer Science and Engineering

Chandpur Science and Technology University

CSE 2206 • Mobile Application Development Project • October 2026